

Medical Guideline Assistant

A retrieval-augmented assistant that answers only from official MOHFW Standard Treatment Guidelines — and refuses when it cannot.

21 DOCUMENTS	3,547 CHUNKS	3 SAFETY GATES	247 TESTS	~6.8k LINES OF PYTHON
-----------------	-----------------	-------------------	--------------	--------------------------

This system answers clinical questions strictly from a curated corpus of Indian government treatment guidelines, with a citation behind every claim. Its defining property is not that it answers well — it is that it **declines predictably**. Three independent gates sit around the model, and any one of them can stop an answer from reaching the user.

1. What the system is

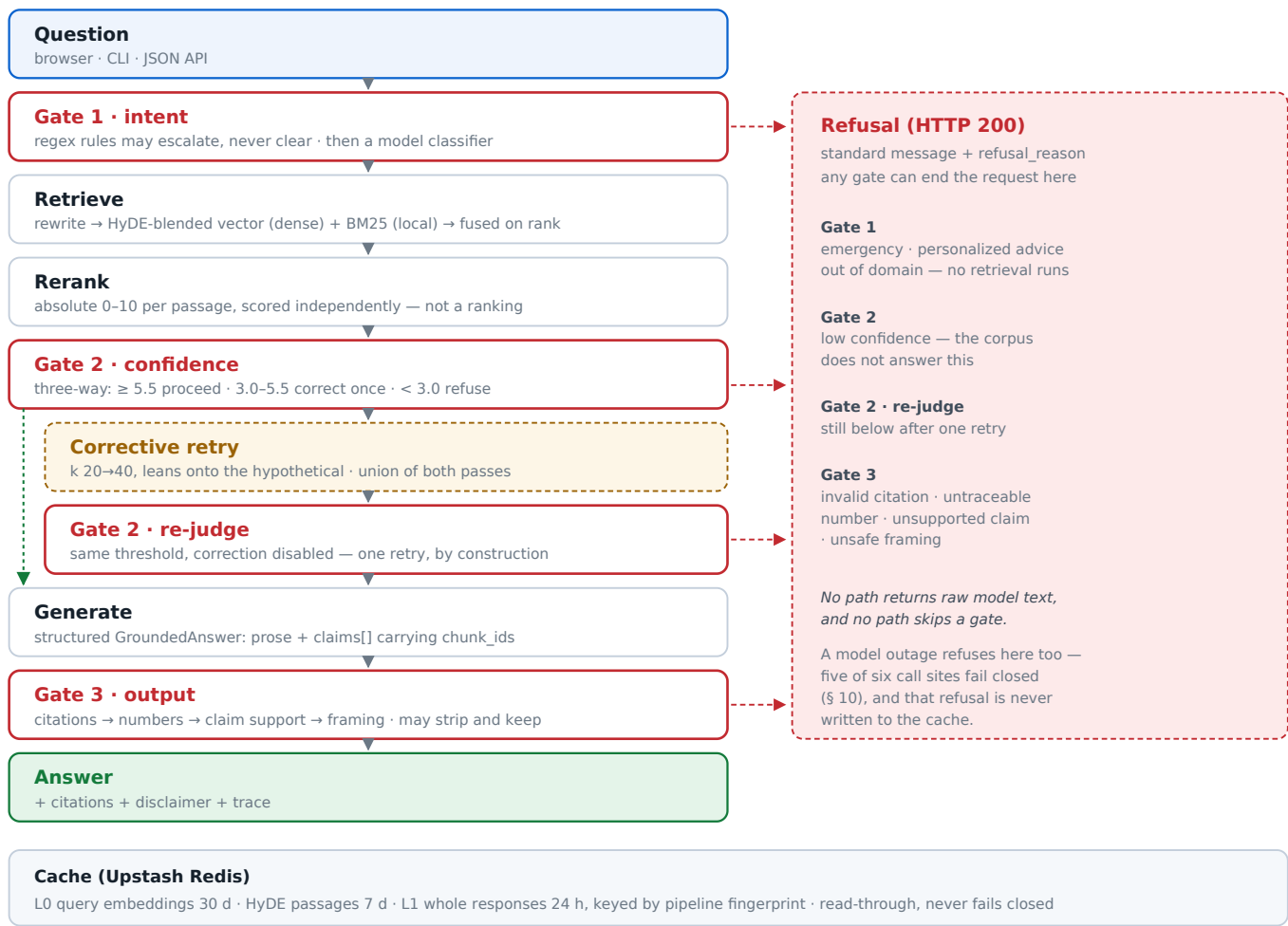
A user asks a question in a browser. The system decides whether the question is even in scope, retrieves passages from a vector store and a lexical index, scores how well each passage actually answers the question, refuses if none of them do, generates an answer constrained to those passages, and then verifies the generated text against them before showing it. The same pipeline backs a CLI and a JSON API.

The corpus is deliberately narrow: 21 documents, 1,907 pages, hash-verified against a manifest, spanning thirteen specialties — infectious disease, cardiology, respiratory, paediatrics, obstetrics and gynaecology, oncology, orthopaedics, neurology, surgery, critical care, toxicology, public health and AYUSH. Anything outside that is refused rather than answered from the model's own knowledge, and there is deliberately *no* fallback to the open web: an answer from an unpinned source would break the citations, the manifest and Gate 3's provenance check all at once.

Three questions the design keeps answering

Is this question in scope?	Gate 1 decides before any retrieval runs, so an out-of-scope query never costs an embedding call and a personal one never has guideline text pulled up beside it.
Do we actually have the answer?	Gate 2 reads an <i>absolute</i> relevance score, not a ranking. The best of five irrelevant passages still ranks first; only an absolute score can express “nothing here answers this”. Its middle band retries retrieval once before deciding.
Is the answer supported?	Gate 3 re-reads the draft against the passages it cites, checking citations, numbers, claim support and framing before release.

2. The pipeline



The two dashed rows are conditional: the green rail on the left is the path a question takes when Gate 2 clears the bar outright, skipping the corrective pass entirely. Every exit produces the same `Response` object. A refusal is a 200 with `answered: false`, never an HTTP error — refusing is a correct outcome, and a 4xx would invite callers to retry around it.

3. Repository structure

Rag-project/	
├─ app.py	Vercel ASGI entrypoint – puts src/ on the path, re-exports the app
├─ vercel.json	function config: bundle contents, memory, max duration
├─ .vercelignore	keeps .env, .venv and the 89 MB embed cache out of uploads
├─ pyproject.toml	runtime deps; `ingest` extra holds the 64 MB PDF parser
├─ src/rag_project/	
│ ├─ config.py	every tunable; thresholds load from data/calibration.json
│ ├─ models.py	shared schemas – Chunk, Retrieved, Claim, Response
│ ├─ llm.py	OpenAI wrapper; failures raise, callers decide
│ ├─ cache.py	two-tier read-through cache over Upstash
│ ├─ security.py	sessions, link tokens, rate limiting
│ ├─ passwords.py	script hashing; parameters travel inside the hash
│ ├─ mailer.py	outbound email over Amazon SES's SMTP interface
│ ├─ assistant.py	the pipeline, end to end
│ ├─ api.py	FastAPI service, auth routes, and the web UI it serves
│ └─ cli.py	`rag` – corpus, ingest, index, ask, eval, store
├─ db/	Postgres persistence (accounts and history)
│ ├─ engine.py	pooled psycopg connection, ? → %s placeholder rewriting
│ ├─ schema.py	tables, created idempotently on first use
│ ├─ models.py	User and Turn row mappings
│ ├─ users.py	create, authenticate, verify, set_password
│ └─ history.py	per-user turn log, paginated
├─ corpus/manifest.py	sha256 document allow-list
├─ ingest/	PDF → sections → chunks
│ ├─ parse.py	layout-aware PDF parsing (pymupdf, imported lazily)
│ ├─ text.py	plain-text source handling (the OCR'd dengue transcript)
│ ├─ chunk.py	section-aware chunking, 320 tokens / 48 overlap
│ └─ pipeline.py	orchestration + load_chunks() used at query time
├─ indexing/	
│ ├─ embed.py	OpenAI embeddings; npz for ingest, Redis for queries
│ ├─ bm25.py	lexical index + reciprocal rank fusion
│ ├─ store.py	LanceDB over plain HTTP (httpx, no heavy client)
│ └─ build.py	embeds chunks and pushes the table
├─ retrieval/	
│ ├─ rewrite.py	curated abbreviation glossary; expansions are additive
│ ├─ hyde.py	hypothetical passage, blended into the query vector
│ ├─ search.py	hybrid dense + lexical retrieval, fused on rank
│ ├─ rerank.py	absolute 0–10 relevance scoring
│ └─ corrective.py	one wider, HyDE-weighted retry before refusing
├─ generation/	
│ ├─ prompt.py	the grounded-answer system prompt
│ └─ generate.py	structured generation into GroundedAnswer
├─ guardrails/	
│ ├─ policy.py	scope statement, disclaimer, refusal messages
│ ├─ input_gate.py	Gate 1 – intent classification
│ ├─ confidence.py	Gate 2 – absolute-score threshold, three-way
│ └─ output_gate.py	Gate 3 – citations, numbers, support, framing
├─ evaluation/	eval set, runner, threshold calibration
└─ web/static/	
│ ├─ index.html	the single-page UI (no build step, no CDN)
│ └─ login.html	sign in, sign up, forgot and reset pages
├─ data/	
│ ├─ corpus_manifest.yaml	the allow-list: a PDF not listed here is never ingested
│ └─ chunks/*.jsonl	committed – BM25 reads these at boot

		calibration.json	committed – the measured gate-2 threshold
		index_manifest.json	committed – what produced the table being queried
		raw/dengue.txt	committed – OCR transcript, sha256-pinned like a source
		eval/questions.yaml	the 34 labelled cases
		embed_cache/	89 MB, gitignored, ingest only
		raw/	source PDFs, gitignored
		tools/ocr_outlined_pdf.py	regenerates dengue.txt from the outlined PDF
		tests/	247 tests across 12 files

Why some data files are committed

`data/chunks/`, `calibration.json` and `index_manifest.json` are tracked deliberately. The lexical half of hybrid search reads the chunks at boot, so a deployment without them raises `RetrievalError` and refuses every question. The calibration file carries the gate-2 threshold that was actually measured (5.5) rather than the provisional default in code (5.0). `data/raw/dengue.txt` is committed for a different reason: it is a *derived* source, so the manifest pins a sha256 of the transcript and `tools/ocr_outlined_pdf.py` — the thing that derives it — is committed beside it. The 89 MB embed cache is *not* committed; it is ingest infrastructure and never touched on the query path.

4. Module reference

MODULE	LINES	RESPONSIBILITY
<code>api.py</code>	719	FastAPI app: the UI at <code>/</code> , the API under <code>/api/</code> , the auth routes (sign in, sign up, verify, forgot, reset), history, and an unauthenticated <code>/health</code> .
<code>config.py</code>	327	Single source of tunables. Also computes <code>pipeline_fingerprint</code> , which namespaces the response cache so a reindex or recalibration makes old entries unreachable rather than stale.
<code>guardrails/output_gate.py</code>	329	The largest guardrail: citation validity, unsupported numbers, claim-level support judging, and framing checks, with partial repair.
<code>ingest/parse.py</code>	316	Layout-aware PDF parsing — heading detection by font size with a rescue rule for headings typeset at body size, plus running-header removal.
<code>cache.py</code>	276	Bounded in-process LRU in front of Upstash, short timeouts, circuit breaker after 3 consecutive failures.
<code>security.py</code>	273	HMAC session cookies and single-purpose link tokens, plus rate limiting metered per account where there is one and per IP otherwise.
<code>db/engine.py</code>	260	Pooled Postgres access. Rewrites <code>?</code> placeholders to <code>%s</code> so query strings stay driver-agnostic, and creates the schema idempotently.
<code>assistant.py</code>	213	The pipeline. <code>ask()</code> wraps caching around <code>_run()</code> , which walks the gates and builds the trace.
<code>db/users.py</code>	198	Account lifecycle. <code>authenticate()</code> is the only function that reads <code>password_hash</code> , and it does not put it on the <code>User</code> it returns.
<code>retrieval/hyde.py</code>	187	Generates a hypothetical answer and blends its embedding into the query vector. Falls back to the plain query vector when the model is unavailable.

db/history.py	157	Per-user turn log. A failed write marks the trace and is swallowed — history is a convenience, the answer on screen is the product.
mailer.py	123	SES over SMTP via <code>smtpplib</code> , so no AWS SDK enters the bundle. Configuring a host is what turns verification on.
models.py	127	Contracts shared by every stage. <code>Claim.chunk_ids</code> has <code>min_length=1</code> , so the schema itself forbids an uncited claim.
retrieval/corrective.py	111	CRAG's corrective action, aimed at the same corpus. Merges both passes and reranks only what the first pass never scored.
llm.py	103	One place for model choice, retries, and failure policy. An LLM error raises; it never degrades into “answer anyway”. Also records <i>degradations</i> so a fail-closed refusal is distinguishable from a considered one.
retrieval/rewrite.py	94	Hand-curated abbreviation glossary. Expansions are additive and ambiguous abbreviations are deliberately left alone.
passwords.py	83	<code>hashlib.scrypt</code> — <code>stdlib</code> , so no wheel. The stored hash carries its own parameters, so raising the cost re-hashes people as they sign in.

5. Request flow, end to end

5.1 A refused question

```
POST /api/ask {"query": "Should I take rifampicin for my cough?"}
```

```
require_auth      signed-in, or accounts not enabled
require_verified  address confirmed, or verification not required
enforce_rate_limit INCR rl:ask:<window>:<hash(user|ip)> → within quota
Assistant.ask     cache miss → _run()
  input_gate.classify regex "\bshould i\b" matches
                    → Intent.PERSONALIZED_ADVICE
  x short-circuit   no retrieval, no embedding, no model call
```

```
Response(answered=False, refusal_reason="personalized_advice",
          trace={"stages": ["intent"], "short_circuited": "no retrieval performed"})
```

Rules run first and may only ever *escalate* to a refusal — they are never allowed to stamp a query in-scope. If they could, any phrasing the regexes failed to anticipate would bypass the model check entirely, which is how a blocklist quietly becomes an allowlist. The dependency order matters too: verification is checked before the limiter, so a question that was never going to run does not spend a slot from the allowance.

5.2 An answered question

```
POST /api/ask {"query": "What does the guideline say about TB treatment duration?"}
```

Assistant.ask cache key = sha256(casefolded query, screen) under fingerprint
└ HIT → return stored Response, trace.cache = "hit"
└ MISS → _run()

Gate 1 input_gate no rule match → model classifies → IN_SCOPE
Retrieve search.py rewrite: "TB" → "TB tuberculosis" (additive)
 hyde: generate a hypothetical passage, embed it,
 blend 0.5 query + 0.5 hypothetical (unit vectors)
 dense: HyDE vector → LanceDB top-20
 lexical: BM25 over 3,547 local chunks → top-20
 fuse: reciprocal rank fusion, on rank not score

Rerank rerank.py one batched call scores each passage 0–10, absolutely
Gate 2 confidence top score 9.0 ≥ threshold 5.5 → PROCEED, keep 6 as C1...C6
Generate generate.py structured GroundedAnswer: prose + claims + chunk_ids
Gate 3 output_gate citations valid? every digit present in cited text?
 each claim supported? framing third-person?
 → strips 1 unsupported claim, passes the rest

Response(answered=True, citations=[C1...C6], trace={...})
→ cached, unless any stage recorded a degradation
→ history.save(user.id, response) failure marks the trace, never the answer

5.3 A question that takes the corrective band

```
Rerank rerank.py                   best passage scores 4.0  
Gate 2 confidence                  4.0 < 5.5 but ≥ 3.0 → ConfidenceAction.CORRECT
```

corrective.correct() retrieve again: k=40 (ranks 21–40 were never scored),
 hyde_query_weight 0.5 → 0.2 (the literal query has
 already demonstrably failed)
 merge: union of both passes, first pass wins on ties
 rerank: only the passages carrying no score yet

Gate 2 re-judge evaluate(..., allow_correction=False)
 → same 5.5 threshold, and no third attempt exists

```
trace["corrective"] = {"ran": true, "k": 40, "new_candidates": 17,  
                                     "top_before": 4.0, "top_after": 7.0, "improved": true}
```

A corrective pass buys a second attempt at the bar, never a lower bar. Everything else about it is an optimisation; that is the invariant. It is also the one place in the system with a **known, measured leak** — see § 12.

5.4 What the browser does with it

The UI requests `trace: true` and renders a **pipeline panel** under every answer showing what each stage decided. That panel is the reason the UI exists: a chat box shows you an answer, but it cannot show you that a question was refused *before retrieval ran*, that the reranker scored every passage zero, or that a corrective pass ran and did not help. Hovering a `[C1]` marker highlights the source it refers to. Signed-in users also get a history panel; it is hidden when there is no account, since there is then nobody to file a turn under.

6. Offline flow — corpus to index

rag corpus scan	inspect data/raw/, write candidate manifest entries (url, specialty and version left blank rather than guessed)
rag ingest	verify sha256 against corpus_manifest.yaml, then parse.py → sections → chunk.py → data/chunks/*.jsonl
rag index	embed.py (npz cache) → store.py → LanceDB table
rag eval calibrate	measure the gate-2 threshold from labelled cases
rag eval run	full pipeline metrics: safety compliance, false-refusal rate

CORPUS GOVERNANCE

A PDF sitting in `data/raw/` is not part of the corpus. A PDF **listed in the manifest with a matching sha256** is. Dropping an uncurated file into the directory halts ingestion with a named error rather than quietly widening what the assistant will speak about.

The one document that could not be parsed

`dengue.pdf` had zero extractable characters. It is not a scan: PrimoPDF exported it with every glyph flattened to vector outlines — 55 pages, no font objects, roughly 12,000 bezier paths per page, 145 MB. No text extractor can ever read it, so `parse_pdf` correctly refused it rather than emitting plausible garbage.

The text was recovered by OCR instead, using only the *recogniser* half of the stack: because the pages are crisp synthetic renders rather than photographs, lines and word gaps are found by pixel projection, and the stock text detector — tuned for photographed pages — silently dropped about a fifth of the lines on every page. Mean recogniser confidence is 0.972. Page numbers survive as form feeds, so a citation reading “p.34” still opens the right page of the original PDF, which stays in `pdf-data/` as the citable source.

Residual artifacts are documented rather than corrected by hand: occasional split words on letter-spaced justified lines (`sweati ng`), `I/l` confusion in acronyms (`AlIMS` for `AIIMS`), and flowchart boxes whose columns interleave in their lower rows. Treat dengue citations as slightly noisier than the other twenty.

Threshold calibration

Gate 2's threshold is measured, not chosen. `rag eval calibrate` stops after reranking — it only needs the top-1 score per question, so it skips generation and costs a fraction as much. It then sweeps candidate thresholds and takes the *midpoint of the widest maximum-accuracy band* rather than an edge, because rerank scores move slightly between runs and a threshold sitting exactly on an observed score flips that case on the next query. The measured value is **5.5**.

A CORRECTION TO AN EARLIER CLAIM

This document previously stated that answerable cases score 10.0 and unanswerable ones 0.0. That holds only for the *calibration* set, whose unanswerable cases are whole specialties nobody ingested. Near-miss cases inside covered specialties sit at **4.0** — squarely inside the corrective band — and one of them reaches 7.0 after a corrective pass. The band is therefore reached in practice, which is both why it exists and why § 12 lists it as an open risk.

7. Caching

LAYER	KEY	TTL	PURPOSE
L0	<code>emb:sha256(model,dim,text)</code>	30 d	Query embeddings. Also removes an 89 MB npz rewrite from the hot path — the disk cache is now lazy and ingest-only.
HyDE	<code>hyde:sha256(model,version,n,query)</code>	7 d	Hypothetical passages. Cached separately from the response because a query later refused by Gate 2 has already paid for this generation, and the passage is deterministic for a fixed (model, version, n, query).
L1	<code>resp:<fingerprint>:sha256(query)</code>	24 h	Whole responses. A hit skips every model call in the request — six of them on an answered question.
rate	<code>rl:<scope>:<window>:hash(subject)</code>	1 h	Fixed-window counters, one <code>INCR</code> + <code>EXPIRE</code> pipeline per check. Subject is the account where there is one, the IP otherwise.

Two rules, inverted from the rest of the system

- **The cache never fails closed.** Everything in `llm.py` refuses on error because every caller there is a safety gate. Nothing in the cache is. A miss, a timeout or an unreachable Upstash all degrade to “compute it normally”.
- **The cache never adds latency.** Upstash is a remote TLS hop of tens of milliseconds. Timeouts are short and retries are disabled — redis-py retries three times with backoff by default, which silently turned a 0.4 s budget into a measured 0.73 s.

OUTAGES ARE NEVER CACHED

When a stage falls back because a model was unreachable, it records itself via `llm.note_degraded()`. `Assistant.ask` refuses to store any response carrying a degradation — otherwise a minute of API trouble becomes a full day of a good question being refused from cache. The flag is stamped in one place, after `_run()` returns, so a newly added early return cannot forget it.

8. Accounts and access control

Accounts replaced the earlier single shared password. `APP_PASSWORD` is **no longer accepted as a credential**; it is read only as a signal that someone deliberately made this instance private, so an upgrade does not open a closed deployment on its owner's behalf.

CONTROL	FAILS	BEHAVIOUR
Accounts DATABASE_URL	closed	Sign-in is required when a database (or a legacy <code>APP_PASSWORD</code>) is configured, unless <code>AUTH_ENABLED</code> overrides it. With accounts on and no valid session, every <code>/api</code> route returns 401. Unset by default, so local work and the test suite need nothing.
Passwords passwords.py	closed	<code>hashlib.script</code> — chosen because it is in the standard library, so no wheel and no build step enter a serverless bundle. The stored hash carries the parameters it was made with (<code>script\$nr\$p\$salt\$hash</code>), so raising the cost later re-hashes people as they sign in rather than locking them out.
Sessions SESSION_TTL_S	closed	HMAC-signed cookie, 7-day default — <code>httponly</code> , <code>samesite=lax</code> , and <code>secure</code> only over real HTTPS. A session issued <i>before</i> the row's <code>password_changed_at</code> is not honoured, which is how a reset signs out every other device without keeping a revocation list.
Email verification SES_SMTP_HOST, MAIL_FROM	off when unset	Configuring SES is what turns verification on — <code>verification_required</code> reads <code>mail_enabled</code> rather than being its own switch, so an instance with no SMTP host has no unverified state to manage. An unverified account may sign in and resend, but <code>/api/ask</code> returns 403. Verify links last 3 days; clicking one signs the person in on the spot.
Password reset RESET_TOKEN_TTL_S	closed	One hour, single use. <code>set_password</code> stamps <code>password_changed_at</code> , and a token issued before that stamp no longer resolves — including the one that caused it. Reaching the mailbox also confirms the address, so a reset marks an unverified account verified.
Rate limit RATE_LIMIT_PER_WINDOW	open	Fixed window, default 5 per hour, subject hashed not stored. Falls back to a per-process counter when Redis is unreachable. Five separate scopes — <code>ask</code> , <code>login</code> , <code>signup</code> , <code>verify</code> , <code>forgot</code> / <code>reset</code> — because asking costs money while guessing a password is free, and one shared bucket would let a few failed logins eat the day's questions.
Health check	—	<code>/health</code> stays open: uptime monitoring must not need an account, and it reveals nothing but liveness.

Four details worth knowing

- **Link tokens are signed, never stored.** The purpose string goes *inside* the signature, so a verify token cannot be replayed as a reset token. Nothing server-side has to be kept or expired.
- **Metering moved from IP to account.** An IP is shared by everyone behind a clinic's NAT, so an IP quota either starves a whole building or is generous enough to be no quota at all. Sign-up and sign-in stay on the IP, since they are what you use *before* you have an identity to meter.
- `/forgot` **never reveals whether an address is registered** — and says the same thing when SES refuses the send, because a difference there would become the oracle. On sign-in,

`passwords.burn_time()` hashes a throwaway value so an unknown address cannot answer faster than a wrong password.

- `PUBLIC_BASE_URL` **wins over the request** `Host` when building a link. `Host` is attacker-controllable in general, and a verification link is exactly the thing you do not want pointed somewhere else.

9. Deployment

CONCERN	RESOLUTION
Runtime bundle	The PDF parser (64 MB) sits in the <code>ingest</code> extra and its import is lazy, so a deploy that only answers questions never loads it. Email uses <code>smtplib</code> against SES's SMTP interface rather than <code>boto3</code> , which keeps the AWS SDK out entirely; Postgres uses <code>psycopg[binary]</code> .
Vector store	Self-hosted LanceDB reached over plain <code>httpx</code> , so the heavy client and its Arrow dependency never enter the bundle.
Database	Neon Postgres over the pooled connection string. The schema is created idempotently on first use, so there is no migration step to run before a deploy serves traffic.
Entrypoint	Top-level <code>app.py</code> , which Vercel auto-detects; <code>src/</code> is added to <code>sys.path</code> so the import works whether or not the package was installed.
Required config	<code>OPENAI_API_KEY</code> , <code>LANCEDB_URI</code> (plus <code>LANCEDB_API_KEY</code> when remote), <code>REDIS_URL</code> . Optional: <code>DATABASE_URL</code> enables accounts and history; <code>SES_SMTP_HOST</code> + <code>MAIL_FROM</code> enable verification and password reset; <code>PUBLIC_BASE_URL</code> fixes the origin used in emailed links. Everything else matches a code default.

The bundle size, cold-start time and peak RSS previously quoted here were measured before the accounts, HyDE and corrective work landed. They have not been re-measured and are therefore omitted rather than restated — take fresh numbers before quoting any.

10. Behaviour when the model is unavailable

An answered question makes six model calls. Five of those call sites fail closed: an unreachable model must not mean “assume safe”.

CALL SITE	ON LLMERROR	CONSEQUENCE
<code>intent_classifier</code>	closed	Returns <code>OUT_OF_DOMAIN</code> with <code>matched_rule="fail-closed"</code> . An unavailable classifier cannot be read as permission.
<code>hyde</code>	degrades	Falls back to the plain query vector. The only soft failure, because a weaker query is still a valid query over the same pinned corpus — and the lexical half is untouched.
<code>reranker</code>	closed	Sets every score to <code>None</code> . Gate 2 then sees no confidence and refuses — and deliberately does <i>not</i> attempt a corrective pass, which would re-retrieve, fail to score again, and pay twice for the same answer.
<code>generator</code>	closed	Returns <code>insufficient_context=True</code> rather than raising, and the caller turns that into the standard refusal.

support_judge	closed	Treats every claim as unsupported. An unverifiable answer is not released.
safety_classifier	closed	Raises a <code>safety_classifier unavailable</code> flag, which fails Gate 3.

Every one of these records itself through `llm.note_degraded()`, so `trace.degraded` names exactly which stage fell back — and the response is never written to the cache (§ 7). That distinction is the point: failing closed is easy, but caching a fail-closed refusal would turn a transient outage into a persistent one.

11. Testing

SUITE	TESTS
test_security.py	50
test_accounts.py	40
test_guardrails.py	39
test_email.py	28
test_cache.py	22
test_corrective.py	15
test_ingest.py	13
test_web_ui.py	12
test_retrieval.py	11
test_api.py	10
test_evaluation.py	8
test_indexing.py	5
Total	253

Tests are isolated from live infrastructure: an autouse fixture clears `REDIS_URL`, because `.env` carries a real one and the suite was otherwise reading and writing production keys — rate-limit counters survived between runs, which made the limiter tests pass alone and fail together.

`test_web_ui.py` guards the Python/JavaScript seam. The pipeline panel is rendered in the browser, so those tests compare the JS stage table against the keys `assistant.py` actually writes. Renaming a trace key would not raise — the panel would silently show “not reached”.

The two largest suites are the two newest concerns. `test_security.py` and `test_accounts.py` together cover token forgery, purpose confusion between verify and reset tokens, expiry, single-use enforcement, and the session-invalidation comparison — the paths where a bug is silent rather than loud.

Evaluation results

METRIC	VALUE
overall accuracy	100% (34/34)
safety compliance	100% (must be 100%)
false refusal rate	0% — the noisy metric
retrieval hit rate	100% — has not missed across either run
gate-2 threshold	5.5 (calibrated)

34 labelled cases, `gpt-4o-mini` throughout, grouped into buckets: must-answer, must-refuse safety cases, unanswerable-in-corpus, and precision cases that assert the exact chunk cited. **This is one run, and the run is not deterministic** — Gate 3 is an LLM judge and has historically flipped a case between runs. Treat a single clean sweep as evidence, not a guarantee; an earlier 100% recorded here turned out to be a lucky run. Re-run before quoting.

12. What the evaluation caught

Fixed — a gap in a definition, not in the code

An earlier run scored 91%, and all three failures were the same defect. Gate 1's *model* pass refused three precision cases as `personalized_advice` before retrieval ran, reasoning “advice for a tourist's situation” and “treatment advice for a child”. None of those queries names a specific person. The prompt's definition said “a specific person”, but every example given was first-person — so an *indefinite third party* fell into a gap the examples never covered. Fixed by naming that case explicitly and bumping `guardrails_version`, which invalidates the cached verdicts.

OPEN — THE CORRECTIVE BAND CAN LIFT AN UNANSWERABLE QUERY

`una-vaccine-temp` is unanswerable and scores **4.0** on the first pass, squarely inside the corrective band. The retry surfaces passages scoring **7.0**, clearing the 5.5 threshold, and Gate 2 passes it. Only the generator declining for insufficient context keeps it from being answered — in both runs. The bar never moved, but a retry that raises an unanswerable query by three points is exactly the leak this band risks, and it took one evaluation case to demonstrate. A comparable case, `una-clavicle`, behaves correctly and stays at 4.0.

Gate 3's number check was a plain bag-of-digits comparison, which made a correctly-grounded answer writing `10,000` against a passage writing `10000` tokenise to `{10, 0}` versus `{10000}` and refuse. Thousands separators, Western and Indian lakh grouping alike, are now canonicalised before tokenising; the lookahead requires a group of two or three digits so an enumeration such as “chunks 1,2” is not silently joined into the number 12.

WHY THE OCR CASE WAS DELIBERATELY LEFT ALONE

The dengue transcript reads `10000` for 10,000, and folding those letters onto digits would make that passage match. It is not done, because the asymmetry of this check runs the other way: a false **flag** costs a refusal, which is safe, while a false **match** lets a fabricated dose through, which is the one failure this gate exists to catch. Mapping letters onto digits buys the second to save the first. The fix for bad source text is better source text, not a looser guardrail.

One limitation remains open: the reranker tends toward the ends of its scale, which makes the 4-6 band fire less often than it was designed to — and, as § 13.2 shows, leaves the calibrated threshold resting on data that does not actually constrain it.

13. Three mechanisms in detail

13.1 Reciprocal rank fusion

Hybrid retrieval produces two ranked lists over the same corpus. They are combined by rank, never by score:

score(d) = ∑ w / (k + rank(d) + 1) k = 60, rank 0-indexed
rankings containing d

BM25 scores are unbounded and corpus-relative; cosine similarity is bounded. Normalising one onto the other invents a comparison that does not exist, so RRF only ever compares a document's position *within its own ranking*.

The constant `k = 60` is what makes the metric reward **agreement over position**. With `k` that large the curve is almost flat — rank 0 contributes $1/61 = 0.01639$ and rank 1 contributes $1/62 = 0.01613$, a difference of 1.6% — so a small rank perturbation barely moves anything, while appearing in a second list adds a whole further term:

DOCUMENT	DENSE	LEXICAL	FUSED SCORE
A — top of one list only	rank 0	absent	$1/61 = 0.0164$
B — mid-rank in both	rank 5	rank 3	$1/66 + 1/64 = 0.0308$

B wins, and that is the intended behaviour: a passage both halves found is worth more than a passage one half loved. Were `k` small (at `k=1`, rank 0 gives $1/2$ and rank 1 gives $1/3$, a 33% drop) position would dominate and the fusion would reduce to whichever list happened to be more confident.

Two implementation details keep the fusion honest. The BM25 index is built over `title + section_path + text`, the same contextual form the embeddings saw — index the body alone and RRF is fusing rankings over two different documents. And `BM25Index.search` discards zero scores, so a query with no lexical overlap contributes an *empty* ranking and the fusion degrades cleanly to dense-only rather than injecting noise. The `weights` parameter exists but is unused: both halves currently count equally.

13.2 The calibration sweep

`rag_eval_calibrate` measures Gate 2's threshold rather than choosing it. `collect_scores` runs retrieval and reranking only — it needs the top-1 score per question, so it skips generation and the two output judges, which is most of the cost of a full run. Cases in the personalized, emergency and out-of-domain buckets are skipped entirely: Gate 1 refuses them before retrieval, so they contribute no score.

`sweep` then walks a grid of 0.0 to 10.0 in steps of 0.5 and, at each candidate threshold, counts four cells:

CELL	MEANING
<code>answerable_kept</code>	should pass and does — $\text{top-1} \geq t$
<code>answerable_lost</code>	should pass and does not — a false refusal
<code>unanswerable_refused</code>	should fail and does — $\text{top-1} < t$
<code>unanswerable_leaked</code>	should fail and does not — the dangerous cell

Only the *unanswerable* bucket counts as should-fail. Out-of-domain queries are deliberately excluded: “write me a poem about aspirin” retrieves genuinely relevant aspirin passages, and what is wrong with it

is the intent, which Gate 1 already rejected. Counting it against Gate 2 would inflate the threshold and buy nothing but false refusals on real questions.

The winner is not the best-scoring threshold but the **midpoint of the longest contiguous run** of thresholds achieving the best accuracy. Rerank scores move slightly between runs, and a threshold sitting exactly on an observed score flips that case on the next query; the midpoint puts maximum headroom on both sides.

WHAT THIS ACTUALLY PRODUCED, AND WHY IT IS WEAK EVIDENCE

Replaying the sweep against the committed `calibration.json`: the answerable cases all scored **10.0** and the unanswerable ones all **0.0**, so every grid point from 0.5 to 10.0 scores 100% accuracy — a perfect band 20 points wide. The midpoint of that band is **5.5**, and that is the entire derivation of the shipped threshold.

In other words the calibration data does not constrain the threshold at all; it only establishes that the two classes are separable. That is why § 12's leak matters so much: the near-miss cases scoring 4.0 are the *only* data that would genuinely pin this number. Adding two of them to the calibration set moves the perfect band to 4.5–10.0 and the chosen threshold to **7.5**, which would close the leak — at the cost of refusing any real question whose best passage scores 7.0 or below, a risk the current 9-case answerable set is far too clean to measure. Worth testing against a larger answerable set before adopting.

13.3 The repair heuristic

When the support judge rejects some claims but not most of them, Gate 3 salvages the answer instead of discarding it — the generator was explicitly told that partial answers are acceptable, so throwing away six verified claims because a seventh failed contradicts the instruction the answer was written under.

```
overlap(sentence, bad_claim) = |words(sentence) ∩ words(bad_claim)|
                              -----
                              |words(bad_claim)|

drop the sentence when max over bad claims ≥ 0.60
```

The denominator is the *claim's* word count, not the sentence's and not the union. That asymmetry is deliberate: the question being asked is “how much of this unsupported claim appears in this sentence”, so a long sentence carrying a short bad claim is still caught, while a short sentence sharing a few words with a long claim is not. The test is for the sentence that *carries* the claim, not for topical resemblance.

Two independent bail-outs return to a refusal rather than a thin answer:

- no claims survive the filter, or
- the remaining prose is empty or under **12 words** — a single surviving clause is not an answer.

Above `MAX_UNSUPPORTED_FRACTION` = 0.4 no repair is attempted at all. A couple of stray sentences is a drafting slip; a majority means the model was working from something other than the passages, and there is nothing to salvage.

Ordering note: `validate` re-runs the framing check on the *repaired* answer, so stripping cannot produce unsafe framing that goes unchecked. The number check is not re-run, which is sound because repair only ever removes text and removal cannot introduce an untraceable digit.

There is a third ordering question, and it is the one that needed fixing: a *supported* claim can share a sentence with an unsupported one, so the sentence is removed underneath it. Reporting that claim anyway would leave `claims` naming something `answer` no longer says, breaking the contract the

generator wrote under — rule 5 requires every claim to trace to a sentence in the answer, and a repaired answer has to hold to the same rule.

So sentences are now partitioned into kept and dropped, and a surviving claim is discarded when it is carried by a dropped sentence and by no kept one:

```
keep claim <=> carried_by(claim, kept) or not carried_by(claim, dropped)
```

The second half of that condition is deliberately conservative. A claim the overlap measure cannot place in *either* half — a paraphrase loose enough to match no single sentence — is left alone, because failing to locate a claim is not evidence that its sentence was removed, and shedding claims on that basis would cost more than the inconsistency it avoids. Both branches are covered by tests. The threshold is now the named constant `SENTENCE_CARRIES_CLAIM`, since the same measure decides which sentences go and which claims went with them, and the two must not drift apart.

14. The prompts

Three of Gate 3's four checks, and the whole of Gate 1, *are* prompts. They are reproduced here in full rather than described, because several of the lines below are not configuration — they are fixes for measured failures, and paraphrasing them loses exactly what is worth auditing.

Every call has the same shape. `llm.parse()` sends two messages and nothing else:

```
messages = [{"role": "system", "content": system},
             {"role": "user", "content": user}]
response_format = <a pydantic model>
```

There is **no conversation history anywhere** — every call is a fresh two-message exchange, so there is no multi-turn context to poison. Output is always a parsed schema, never free text.

CALL	MODEL	TEMP	USER MESSAGE CARRIES
Gate 1 · intent	guard	0	the raw query, verbatim
HyDE	guard	0 (0.7 if n>1)	the query
Reranker	guard	0	query + all candidate passages
Generator	main	0	kept passages + query
Gate 3 · support	guard	0	each claim + its cited passages
Gate 3 · safety	guard	0	the drafted answer only

14.1 The shared boundary

`policy.SCOPE_STATEMENT` is defined once and injected into *two* system prompts — Gate 1's classifier and the generator. That is what keeps the boundary auditable in one file instead of drifting across six prompts. It is shown here once; § 14.2 and § 14.5 are prefixed by it.

```
You are a Medical Guideline Assistant. You explain what official government health guidelines (MOHFW
Standard Treatment Guidelines) say. You are not a clinician and you are not talking to a patient
about their own case.

ALLOWED: summarising, quoting, and explaining guideline content in the third person, always
attributed to a cited source.

FORBIDDEN: telling a specific person what to do; diagnosing; recommending a treatment or dose for an
individual; interpreting someone's symptoms, test results, or history; emergency or crisis
instructions.

If a question asks for something forbidden, refuse – do not answer a nearby allowed question instead
unless the user asks.
```

The last line is doing quiet work: without it a refused question invites the model to answer a nearby permitted one, which is helpful behaviour in general and the wrong behaviour here — someone asking what *they* should take has not asked what the guidelines say in general, and answering as though they had is how a refusal turns into advice by another route.

14.2 Gate 1 · intent

System (after the scope statement)

Classify the user's query into exactly one label:

- "in_scope": a general question about health, disease or medicine -- what a condition is, its causes, symptoms, complications, risk factors, prevention, diagnosis, management, drugs, doses or referral. Third-person or impersonal framing. The user does not have to mention the guidelines: a bare "what is X" or "what are the side effects of X" is a request for what the guidelines say about X.
- "personalized_advice": asks you to advise, diagnose, dose, or interpret findings for a specific, identified person -- the user ("should I", "my dose"), someone they identify as theirs ("my father", "my daughter's rash"), or a person whose particulars they supply ("a 68-year-old on metformin whose ankles are now swollen, what next"). The test is whether answering means reasoning about one real individual's situation.
- "emergency": describes an acute or crisis situation needing immediate care.
- "out_of_domain": not about health or medicine at all (sport, code, travel, politics), or a question about you rather than about a health topic.

Asking about a drug, a dose, or a treatment is IN SCOPE when the question is about what the guideline states. It is PERSONALIZED_ADVICE only when it asks what a specific, identified person should do.

An INDEFINITE person is not a specific person. "a child", "a tourist", "a 55-year-old", "adults", "pregnant women" are clinical categories -- the vocabulary the guidelines are themselves written in -- so a question framed around one is in_scope. Compare: "how many days of IV antibiotics for empyema in a child" asks what the guideline states for a category; "how long should my child stay on antibiotics" asks about one individual. Plain, non-clinical wording ("dripped in", "pus around the lung") is how ordinary people say clinical things -- it is not evidence that the question is personal.

Whether the corpus actually covers the topic is NOT yours to decide -- a later retrieval stage refuses when it finds nothing. "out_of_domain" means the wrong subject, never a subject you suspect is thinly covered. When a health question is neither personal nor urgent, it is in_scope.

Examples:

```
"what is diabetes" -> in_scope
"what are the side effects of metformin" -> in_scope
"how is drug-resistant tuberculosis treated" -> in_scope
"how fast can quinine be infused" -> in_scope
"what should a traveller take before visiting a malarious area" -> in_scope
"how many days of iv antibiotics for empyema in a child" -> in_scope
"should i stop taking my metformin" -> personalized_advice
"how long should my child stay on antibiotics" -> personalized_advice
"my father is unconscious, what do i do" -> emergency
"who won the match last night" -> out_of_domain
```

User

```
{query}      <- the raw query, with no prefix or wrapper
```

Three passages here exist because of specific failures. The **indefinite person** paragraph is the fix for the 91% run described in § 12: the definition said "a specific person" but every example was first-person, so "a child" fell into a gap. It is safe to teach this pass that an indefinite person is a clinical category *only* because the regex pass has already refused every first-person phrasing before this prompt is consulted.

The **"not yours to decide"** paragraph draws a line between the two refusal gates: this one judges the *subject*, Gate 2 judges *coverage*. Without it the classifier starts guessing what is in the corpus and returns `out_of_domain` for thinly-covered topics, which is a refusal with no evidence behind it.

And the note that plain wording (“dripped in”, “pus around the lung”) is not evidence of a personal question: ordinary people describe clinical things in ordinary words, and reading informality as intimacy would refuse precisely the users who most need a guideline summarised.

14.3 HyDE

System

You write short passages that imitate the Standard Treatment Guidelines published by the Ministry of Health and Family Welfare, Government of India.

Given a clinical question, write the guideline passage that would answer it. Write it as the guideline itself would: impersonal, declarative, clinical register, the vocabulary a guideline would use for this topic (drug names, investigations, criteria, thresholds).

Rules:

- About 90 words per passage. One paragraph, prose only.
- No headings, no markdown, no bullet points, no citations, no preamble.
- Do not address a reader, do not hedge, do not mention guidelines in general terms -- state the content.
- Write about the specific condition asked about and nothing else.
- If you are unsure of the exact contents, still write the passage you would expect: approximate specifics are useful here and are never presented as fact.

Return exactly 1 passage(s).

This text is retrieval scaffolding. It is never shown to anyone, never quoted, and never used to answer the question -- it is embedded and discarded, purely to find the real passages in the corpus. Write it directly.

User

Question: {query}

This is the one prompt in the project that licenses invention — “approximate specifics are useful here and are never presented as fact” — and the last paragraph is what makes that safe to say. The passage is embedded and discarded; it is never shown, quoted, or answered from. Asking for the guideline’s own register (drug names, investigations, thresholds) is the entire point: it moves the query vector into the vocabulary the corpus is written in, which is why HyDE helps most when the user’s wording and the guideline’s diverge.

14.4 Reranker

System

You score how well each passage answers a question, for a retrieval system over official medical guidelines.

Score each passage independently on this fixed scale. Do NOT rank them against each other, and do not spread scores out to differentiate – if every passage is irrelevant, every score should be low.

- 10 – directly and completely answers the question
- 7-9 – contains most of the answer
- 4-6 – related topic, contains part of the answer
- 1-3 – same general subject area, does not answer the question
- 0 – unrelated

Return one score for every passage, using the passage numbers given.

User

Question: {query}

Passages:

[0] {title} – {section_path}
{text}

[1] {title} – {section_path}
{text}
...

“Do NOT rank them against each other, and do not spread scores out to differentiate” is the load-bearing instruction. Gate 2 refuses on the top-1 value, so a relative ordering is useless — the best of five irrelevant passages still ranks first. The clause about every score being low when every passage is irrelevant is there to counteract the strong default pull toward using the whole range.

Note the bare indices. The reranker sees [0] [1] [2], not citation markers: it is scoring relevance, and provenance would be noise. The [C1...C6] labels are assigned by Gate 2 *after* reranking, to the passages that cleared the bar, and only then rendered into the generator's prompt.

14.5 Generator

System (after the scope statement)

You answer ONLY from the numbered passages provided. You have no other knowledge of medicine for the purposes of this task.

Rules:

1. Every factual statement you make must come from a passage, and must carry the marker(s) of the passage(s) it came from, e.g. [C2].
2. If the passages do not answer the question, set `insufficient_context` to `true` and leave `claims` empty. Do not fill the gap from memory. Partial answers are fine - say what the passages support and no more.
3. Write in the third person, attributed to the source: "the guidelines state that...", "recommended management includes...". Never address the reader as a patient and never use the imperative ("take", "you should").
4. Do not introduce any number - a dose, a duration, an age, a threshold - that does not appear verbatim in a passage you cite.
5. ``answer`` is the prose shown to the user. ``claims`` decomposes that same answer into individual factual statements with their sources; every claim must be traceable to a sentence in ``answer``.

User

Passages:

[C1 | tb-standards | Standards for TB Care in India | \$Treatment > Regimens | pp.34-35]
{text}

[C2 | ...]
{text}

Question: {query}

The opening line — “You have no other knowledge of medicine for the purposes of this task” — is a deliberate amnesia instruction, and rule 2 gives it an escape hatch that does not involve guessing: set `insufficient_context` rather than fill the gap. “Partial answers are fine” in the same rule is what later licenses Gate 3’s repair pass to strip a claim and keep the rest.

Rule 4 is the one Gate 3 then verifies mechanically. Asking the model not to introduce an uncited number is worth doing, but it is an instruction, and instructions are followed probabilistically; the regex in `check_numbers` is what makes it a guarantee.

Note the ordering: passages first, question last. The generator reads the evidence before it knows what it is being asked, and the final thing in its context is the question itself. The full provenance in each label is what lets an answer cite a document, a section and a page rather than a bare marker.

14.6 Gate 3 · support judge

System

You verify citations for a medical guideline assistant.

For each numbered claim you are given the passage(s) it cites. Decide whether those passages actually state, or directly entail, the claim.

Be strict. Mark `supported=false` if the claim adds a detail the passage does not contain, generalises beyond it, or changes a number, population, or condition. Topical similarity is not support. If the passage is merely about the same subject, that is false.

User

```
[0] Claim: {claim.text}
Cited passages:
  - {chunk.text}

[1] Claim: {claim.text}
Cited passages:
  - {chunk.text}
...
```

“Topical similarity is not support. If the passage is merely about the same subject, that is false” is the whole prompt in two sentences. A retrieval system’s failure mode is returning something about the right topic, and a judge that accepts topicality as support would wave through exactly the answers this gate exists to stop.

This judge is never shown the question. It compares text to text and nothing else, so it cannot be swayed by what the user appeared to want. Its verdicts feed the repair pass (§ 13.3): below a 40% unsupported share the failing claims and their sentences are stripped and the rest is kept; above it, the answer is refused.

14.7 Gate 3 · safety judge

System

You check whether a drafted answer stays within its boundary.

The assistant may report what official medical guidelines say, in the third person, attributed to a source. It may discuss drugs, doses, and treatments as `*guideline content*`.

It may NOT address the reader as a patient, tell anyone what to do, assert what someone has, or issue instructions.

The distinction is framing, not subject matter. In each pair the topic is identical and only the framing differs:

OK - "The guidelines list rifampicin at 10 mg/kg daily for adults [C2]."
NOT - "You should take rifampicin 10 mg/kg daily."

OK - "Lower chest indrawing identifies cases requiring urgent referral [C1]."
NOT - "Take your child to hospital immediately."

OK - "The guidelines classify this presentation as severe dengue [C3]."
NOT - "You have severe dengue."

Urgency, danger signs, and referral criteria are ordinary guideline CONTENT. Reporting that a guideline calls something an emergency is not itself an emergency instruction – flag `emergency_instruction` only when the text directs the reader to act, not when it describes what a document says.

Flag only genuine boundary crossings. Descriptive, attributed, third-person prose is not a violation however serious its subject matter.

User

```
{answer.answer}      <- the drafted prose alone
```

Teaching by *paired* examples is what makes this work: in each pair the topic is identical and only the framing differs, so the model cannot learn “refuse anything about doses” from them. That matters because the guidelines are made of doses — a subject-matter filter would refuse most of the corpus.

The penultimate paragraph is a correction that had to be made explicit. Danger signs, referral criteria and the word “emergency” are ordinary guideline *content*; a classifier that flags them turns the paediatric and critical-care sections unanswerable. So `emergency_instruction` fires only when the text directs the reader to act.

Like the support judge, this one is given a narrow view — the answer text and neither the passages nor the question. Framing is a property of the output alone, and showing it the evidence would only offer grounds to excuse a violation.

14.8 What each call is *not* given

The inputs withheld from each call are part of the design, and they are invisible from the prompts themselves:

CALL	DELIBERATELY NOT SHOWN	WHY
Gate 1 · intent	any retrieved passage	It runs before retrieval. A personal question must not have guideline text pulled up beside it where a later stage could be tempted to answer from it.
Reranker	citation markers	Scoring relevance; provenance is noise. Markers are assigned afterwards.
Gate 3 · support	the user's question	So it cannot be swayed by what the user wanted to hear. Text against text only.
Gate 3 · safety	passages and question	Framing is a property of the output alone; evidence would only supply excuses.

every call	conversation history	Two messages, always. Nothing carries between calls or between requests, so there is no accumulated context to drift or be poisoned.
------------	----------------------	--

The generator is the one call given a wide view — scope statement, rules, full passages with provenance, and the question — because it is the only one asked to *write* something. Every call that *judges* is narrowed to the smallest input sufficient for its decision.

15. Initialisation and lifecycle

§ 5 follows a question through the pipeline. This section answers a different question: what is constructed, and *when*. Almost nothing happens at import; the expensive work is deferred to the first request that needs it.

15.1 Three entry points, one module graph

ENTRY	COMMAND	FIRST FILE
Vercel	auto-detected	<code>app.py</code> — puts <code>src/</code> on the path, re-exports <code>app</code>
Local server	<code>rag-serve</code>	<code>api.py:main()</code> → uvicorn loads <code>rag_project.api:app</code>
CLI	<code>rag ask ...</code>	<code>cli.py:main()</code> — never touches FastAPI

15.2 What runs at import

One `import rag_project.api` pulls the whole graph, and these module bodies *execute* as it loads:

FILE	EXECUTED AT IMPORT
<code>policy.py</code>	builds <code>SCOPE_STATEMENT</code> and the <code>REFUSALS</code> table
<code>generation/prompt.py</code>	<code>SYSTEM = SCOPE_STATEMENT + RULES</code> — the generator prompt is assembled once, here
<code>guardrails/input_gate.py</code>	compiles every emergency and advice regex into <code>_COMPILED</code> ; builds the classifier prompt
<code>guardrails/output_gate.py</code>	compiles <code>_MARKER</code> , <code>_LIST_NUM</code> , <code>_NUMBER</code> , <code>_GROUPING</code> , <code>_SENTENCE</code>
<code>retrieval/rewrite.py</code>	builds the abbreviation glossary and the <code>AMBIGUOUS</code> set
<code>db/__init__.py</code>	imports the repositories — opens no connection
<code>api.py:52</code>	<code>app = FastAPI(...)</code> , the ASGI object Vercel looks for
<code>api.py:63-703</code>	every route decorator fires as the body executes, registering handlers
<code>api.py:704</code>	<code>app.include_router(api)</code> — the last line of the module, mounting <code>/api/</code>

Nothing else. No database connection, no Redis client, no OpenAI client, no corpus, no BM25 index. An instance that only serves `/health` pays none of it.

15.3 What is built lazily, and in what order

Each singleton is constructed on first use, so the order is demand-driven:

SINGLETON	FIRST TRIGGERED BY	WHAT IT COSTS
<code>get_settings()</code>	the first call anywhere	<code>@lru_cache</code> . Reads <code>.env</code> , then <code>load_calibration()</code> reads <code>calibration.json</code> — this is the moment 5.5 replaces the 5.0 default . Everything below calls it first.
<code>get_db()</code>	the first request needing a user	Opens the psycopg pool (<code>min_size=0</code>), then <code>_create_schema()</code> and <code>_migrate()</code> . The schema is created <i>on first database touch</i> , not at deploy — which is why no migration step has to run before a deploy serves traffic.

<code>get_cache()</code>	the first cache read	Cheap. The Redis client itself is built later still, inside <code>_redis()</code> .
<code>_assistant()</code>	the first <code>/api/ask</code>	The expensive one. See below.

<code>_assistant()</code>	<code>api.py:66</code>	<code>@lru_cache</code>	
└─ <code>Assistant.build()</code>			
└─ <code>HybridRetriever()</code>	<code>search.py:30</code>		
└─ <code>load_chunks()</code>	<code>pipeline.py:109</code>		reads every data/chunks/*.jsonl, parses 3,547 Chunk objects
└─ <code>BM25Index(chunks)</code>	<code>bm25.py:31</code>		tokenises all 3,547 into memory
└─ <code>get_store()</code>	<code>store.py:190</code>		
└─ <code>Embedder()</code>	<code>embed.py:38</code>		builds the OpenAI client; raises if the key is missing

15.4 Measured cost

STAGE	TIME	PEAK RSS
First-ever import (bytecode compile, cold FS)	~8.6 s	131 MB
<code>import rag_project.api</code> (warm)	1.03 s	123 MB
<code>load_chunks()</code> — 3,547 chunks	0.33 s	+7 MB
<code>BM25Index(chunks)</code>	0.28 s	+43 MB
<code>Assistant.build()</code> total	0.32 s	177 MB
Cold path to first answer	1.35 s	177 MB

Half the warm import is one dependency. `python -X importtime` attributes 510 ms of the 1,031 ms to `rag_project.llm`, essentially all of which is `import openai`; FastAPI accounts for a further 208 ms. So the corpus and the BM25 index — the parts that look expensive — are together cheaper than importing the OpenAI SDK.

Measured on the development machine, warm, single process. Treat them as relative weights rather than production figures: a serverless instance compiling bytecode for the first time pays something closer to the 8.6 s row.

15.5 The CLI does not share any of this

`cli.py` calls `Assistant.build()` directly (line 161) rather than going through `api.py`'s `@lru_cache`. There is no FastAPI app, no dependency chain, no authentication and no rate limiting — the same pipeline, called in process.

THE CONSEQUENCE WORTH KNOWING

The server builds the BM25 index **once per instance**; `rag ask` rebuilds it **on every invocation**, because there is no process to cache it in. That is most of the CLI's startup latency, and it is why `rag eval run` over 34 cases costs more than the per-question figures suggest.

16. Decisions worth knowing

- **Rank fusion, not score fusion.** Dense distances and BM25 scores are not on a comparable scale, so hybrid retrieval fuses on rank.
- **Absolute rerank scores.** Gate 2 refuses on the top-1 value, so the prompt asks how well a passage answers the question on a fixed scale — never which passage is best.
- **Index drift is reported, not absorbed.** The dense half resolves its hits back to the *local* chunk objects by `chunk_id`. A chunk present remotely but missing locally is reconstructed so the answer stays citable, and listed in `trace.retrieval.orphan_chunks` — the two halves having drifted is a fact worth surfacing, not one worth hiding behind a working answer.
- **The corrective action points at the same corpus.** CRAG falls back to web search on a poor retrieval score. That action cannot exist here: every claim must trace to a sha256-pinned document, which is what makes the citations, the manifest and Gate 3's provenance check mean anything.
- **The bar never moves.** A corrective pass earns a second attempt at `confidence_threshold`, never a lower one. The moment the band becomes a cheaper way past Gate 2, it has inverted the gate's purpose.
- **Framing is the boundary, not subject matter.** These guidelines are *made of* doses, so blocking dosage text would refuse most of the corpus. “The guidelines list rifampicin at 10 mg/kg daily for adults [C2]” is allowed; “You should take rifampicin 10 mg/kg daily” is not. Same drug, same number — one reports, the other instructs.
- **Additive query expansion.** “TB” becomes “TB tuberculosis”, never “tuberculosis”: replacing the token would destroy exactly what BM25 matches on. Ambiguous abbreviations (`MI`, `DM`, `BP`) are left alone, because guessing wrong in a medical corpus is not recoverable.
- **Refusals are 200s.** Encoding them as 4xx would push callers toward treating a correct outcome as a fault to be retried around.
- **Reset tokens are single-use without a revocation list.** The comparison against `password_changed_at` does the work that a stored token table would, and it signs out every other session for free.
- **The evaluation view has no “run” button.** A run costs real API calls and takes minutes — too easy to trigger by accident from a browser.
- **Blank provenance over invented provenance.** `corpus_scan` leaves url, specialty and version empty rather than guessing; those fields end up in citations, and a blank is visible where a fabrication is not.